

HOMEWORK

Overview

This project requires you to build a simplified data manipulation library called **FooDataFrame** that demonstrates advanced Python concepts including Object-Oriented Programming, iterators, generators, and decorators. You will implement two main classes (**Column** and **FooDataFrame**) to work with tabular data stored in CSV files.

CSV File Format

The CSV files used in this project have a specific format:

Header row: `columnname:type,columnname2:type,...`

Data rows: Values separated by commas

Example:

```
name:str,age:int,salary:int
Alice,25,50000
Bob,30,60000
Charlie,35,75000
```

Important assumption: You may assume that the CSV file is never empty and has a correct format. This means:

- The header format is correct
- No `None` or missing values
- Each column is either entirely `int` or entirely `str`
- All values are valid for their declared type
- All rows have the same number of columns

Note on Validation: Although these assumptions simplify CSV parsing, you **must still validate input** when creating Column objects directly. This is necessary because FooDataFrame (see below) provides methods like `add_column()` that allow columns to be added programmatically, not just parsed from CSV. Therefore, the Column constructor (see below) must validate all input parameters to ensure data integrity.

Part 1: Column Class

Attributes (Internal - use underscore prefix)

The Column class must maintain at least the following internal attributes:

- `_name`: Column name (string)
- `_dtype`: Data type, either `"int"` or `"str"`

- **_values**: List containing the column's values
- **_length**: Number of elements in the column

Important: All attributes must remain internally consistent. For example, **_length** must always match `len(_values)`.

Constructor

```
Column(name, dtype, values)
```

Parameters:

- **name** (str): The column name
- **dtype** (str): Either `"int"` or `"str"`
- **values** (list): List of values for the column

Validation: The constructor should validate the input and raise appropriate exceptions:

- **TypeError**: If types are incorrect:
 - **name** is not a string
 - **dtype** is not a string
 - **values** is not a list
- **ValueError**: If values are invalid:
 - **dtype** is neither `"int"` nor `"str"`
 - **dtype** is `"int"` but values contain non-integers.
 - **dtype** is `"str"` but values contain non-strings.

Getter Methods

Implement at least the following getter methods to access internal attributes:

- **get_name()** → returns column name
- **get_dtype()** → returns column data type
- **get_values()** → returns list of values
- **get_length()** → returns number of elements

Element Access

Implement functionality to access individual elements:

- Either use special methods (dunder methods like `__getitem__`) to enable `col[i]` syntax
- Or implement a custom method **get_element(index)**

The result should allow accessing the *i*-th element of the column. If the index is not valid then an **IndexError** should be raised.

Statistical Methods

Implement some statistical methods. The following are mandatory, but you can add other methods:

- `sum()` – Returns the sum of all values (only for `int` columns)
- `mean()` – Returns the arithmetic mean (only for `int` columns)
- `count()` – Returns the number of elements (works for both types)
- `min()` – Returns the minimum value (works for both types)
- `max()` – Returns the maximum value (works for both types)
- `unique()` – Returns a list of unique values (works for both types)

Validation: Each method must validate that it's being called on an appropriate column type. For example, calling `sum()` on a `str` column should raise a `TypeError` with a descriptive message.

Implementation note: You have freedom in how these are computed - either calculate on-demand when invoked, or maintain updated values in additional attributes if the column can be modified.

String Representation

Implement the `__str__()` method to provide a readable string representation of the column, displaying its name, data type, and contents in a user-friendly format. This allows you to print a Column object using `print(col)`.

Part 2: FooDataFrame Class

Attributes (Internal - use underscore prefix)

The FooDataFrame class must maintain at least the following internal attributes:

- `_size`: Total number of data rows in the DataFrame
- `_shape`: A tuple (`num_rows`, `num_cols`) representing DataFrame dimensions
- `_column_names`: A list of strings containing all column names in order
- `_columns`: A dictionary with column names as keys and Column objects as values

Critical requirement: All attributes must be internally consistent at all times:

- All Column objects must have the same `_length`
- `_size` must equal the number of rows
- `_shape[0]` must equal `_size`
- `_shape[1]` must equal the number of columns
- `_column_names` must match the keys in `_columns`

Constructor

```
FooDataFrame(filepath)
```

Parameters:

- `filepath` (str): Path to a CSV file

Behavior:

1. Parse the CSV file from the given path
2. Extract column names and types from the header row (format: `name:type, name:type, ...`)
3. Create Column objects for each column with the appropriate name, dtype, and values
4. Populate the `_columns` dictionary
5. Set all internal attributes (`_size`, `_shape`, `_column_names`) consistently

Validation: The constructor needs to perform the following:

- Raise `FileNotFoundError` if the file doesn't exist

Recall that, according to the assumptions in the **CSV File Format section** (see above), if the file exists then its format is correct and contains no `None` values.

Getter Methods

Implement at least the following getter methods:

- `get_shape()` → Returns the shape tuple (`num_rows`, `num_cols`)
- `get_size()` → Returns the number of rows
- `get_column_names()` → Returns the list of column names
- `get_column(column_name)` → Returns the Column object for the given column name (raise `KeyError` if column doesn't exist)

Add Column

```
add_column(name, dtype, values)
```

Adds a new column to the DataFrame.

Parameters:

- `name` (str): The new column name
- `dtype` (str): Data type, either `"int"` or `"str"`
- `values` (list): List of values for the new column

Validation (in this specific order):

1. Raise `ValueError` if a column with the given `name` already exists in the DataFrame.
2. Raise `ValueError` if the length of `values` doesn't match the DataFrame's current size (`_size`).
3. Create a Column object with the provided parameters. Any `TypeError` or `ValueError` raised by the Column constructor will automatically propagate to the caller.
4. If all validations pass, add the column to `_columns`, append the name to `_column_names`, and update `_shape` to reflect the new number of columns.

String Representation

Implement the `__str__()` method to provide a readable string representation of the DataFrame in a tabular format where each printed line represents one row. Include column names and show data aligned properly for readability. This allows you to print a DataFrame using `print(df)`.

Example format:

name	age	salary
Alice	25	50000
Bob	30	60000
Charlie	35	75000

Part 3: Optional DataFrame Methods

The following methods are **optional**. If you choose to implement these methods, you must ensure that all object attributes remain internally consistent.

Append Row

```
append_row(row_dict)
```

Appends a new row to the end of the DataFrame.

Parameters:

- **row_dict** (dict): Dictionary mapping column names to values, e.g., `{"name": "David", "age": 28, "salary": 55000}`

Validation (in this specific order):

1. Raise **ValueError** if the keys in **row_dict** do not exactly match the column names in the DataFrame (check both missing and extra keys).
2. Raise **TypeError** if any value in **row_dict** has a type that doesn't match the corresponding column's dtype:
 - For an **"int"** column, the value must be of type **int**
 - For a **"str"** column, the value must be of type **str**
3. If all validations pass, append the values to the corresponding columns, increment **_size**, and update **_shape** to reflect the new number of rows.

CRITICAL IMPLEMENTATION REQUIREMENT: You must validate ALL values in **row_dict** **BEFORE** appending any value to any column. This ensures that if a validation fails (e.g., wrong type for one column), the DataFrame remains in a consistent state with all columns having the same length. Only after all validations succeed should you proceed to append values to the columns.

Implementation Suggestion: Consider implementing an auxiliary method in the Column class (e.g., **append_value(value)**) that is responsible for appending a single value to that column. This method can handle type validation and maintain column consistency. Then, **append_row()** can validate all values first, and only if everything is valid, call this auxiliary method for each column.

Remove Last Row

```
remove_last_row()
```

Removes the last row from the DataFrame. If the DataFrame is empty, raise **ValueError**.

Implementation Suggestion: Consider implementing an auxiliary method in the Column class (e.g., `remove_last()`) that removes the last element from a single column. Then `remove_last_row()` can call this method on each column to keep all columns consistent. If consistency is managed correctly and no exception is raised, `remove_last()` can be executed safely on each column.

Remove Row by Index

```
remove_row(index)
```

Removes the row at the specified index.

Parameters:

- **index** (int): Row index (0-based, must be between 0 and size-1)

Raise **IndexError** if **index** is not valid.

Implementation Suggestion: If consistency is managed correctly and no exception is raised, you can safely use an auxiliary method in Column (e.g., `remove_by_index(index)`) to remove the element from each column.

Part 4: RowIterator Class

Create a **RowIterator** class that implements the Iterator Protocol to iterate over DataFrame rows.

Constructor

```
RowIterator(dataframe)
```

Parameters:

- **dataframe**: A FooDataFrame instance

Iterator Protocol

Implement the Iterator Protocol:

- `__iter__(self)` → Returns **self**
- `__next__(self)` → Returns the next row, raises **StopIteration** when exhausted

Row Format

Each iteration should return a **list of values** for that row, respecting the order of column names as defined in the DataFrame's `_column_names` attribute.

Example (assuming columns are ordered as `["name", "age", "salary"]`):

```
["Alice", 25, 50000]
```

Usage Example

```
df = FooDataFrame('data.csv')
row_iter = RowIterator(df)

for row in row_iter:
    print(row) # Prints: ['Alice', 25, 50000], ['Bob', 30, 60000], etc.
```

Part 5: Generator Functions

Implement the following generator functions:

Column Generator

```
def columns_generator(dataframe):
    """
    Generator that yields information about each column.

    For each column, yield a tuple: (name, dtype, values).
    """
```

Behavior: For each column in the DataFrame, yield a tuple `(name, dtype, values)` using `yield`. In `main.py`, iterate over the generator and print the yielded tuple.

Statistics Generator (Optional)

```
def stats_generator(dataframe):
    """
    Generator that yields statistical information for each column.

    For numeric columns, compute and display statistics (count, sum, mean,
    min, max).
    For string columns, show appropriate statistics (count, unique values).
    """
```

Behavior: For each column, build a string with the relevant statistics and `yield` that string. In `main.py`, iterate over the generator and print each yielded string.

Part 6: Decorators

Implement a decorator to log function calls. You may use the `log_function_call` decorator from the second week's lessons, or implement your own custom decorator. Place the decorator inside the `Column` class file (see below `column.py`).

Task: Apply the decorator to the `unique()` method of the `Column` class to log each time it is called.

Requirements:

- The decorator should not alter the method's return value or behavior
 - If using your own decorator, ensure it is well-documented.
 - The logging output should be informative and follow a consistent format
-

Submission

Project Structure (MANDATORY)

For evaluation purposes, your project MUST be organized in the following way:

Your submission must contain the following files with the exact names and organization:

```
your_submission/
├── column.py           # Column class ONLY
├── dataframe.py       # FooDataFrame class ONLY
├── generators.py      # Generator functions ONLY
├── iterators.py       # RowIterator class ONLY
├── main.py            # Demonstration code
├── data.csv           # Sample CSV
└── documentation.(txt,pdf,md,...) # Your library documentation
```

`main.py` must include a small, runnable example of how to use your library (load a CSV, access columns, iterate rows, and show any optional features). `documentation` should be a concise high-level guide explaining what your library does, how to use it, and any implementation choices that differ from this specification. You can provide a `.txt`, `.pdf`, `.md`, or any readable file format.

IMPORTANT: Each class and set of related functions must be in its own file as specified above. This structure is required because the evaluation system will import and test each module separately.

How to Submit and Deadline

0. **The submission deadline is March 15 at 11:59 PM (UTC +1).**
1. Assuming your matricola is `123456`, create a folder named `homework_123456`
2. Place all files listed above inside this folder
3. Compress the folder as `homework_123456.zip`

4. We will provide details on the submission method in next weeks

Grading

Grading Scale: 0 to 2 Points

Your submission will be evaluated automatically and manually using a standardized test suite. The grading scale is:

- **1 point (maximum):** Earned by correctly implementing the **basic required functionality**, following the **required project structure**, and providing **sufficient code quality** (clear organization and readable code).
 - Your code must work with any CSV file that meets the specified requirements and respects the stated assumptions. Evaluation tests may use different CSV files.
 - **2 points (maximum):** Earned only by implementing the **optional methods/functions** and providing **rigorous, well-structured code comments** throughout the project
-

Important Constraints

You must implement everything from scratch using only Python's built-in features. This means:

- × **NO** external libraries like **pandas**, **numpy**, or any data manipulation libraries
- ✓ **YES** to built-in Python types: **list**, **dict**, **str**, **int**, **tuple**
- ✓ **YES** to built-in functions: **open()**, **len()**, **sum()**, **min()**, **max()**, etc.

The goal is to spend time coding, thinking about how to combine small pieces into a cohesive project, and demonstrating your understanding of Python fundamentals and advanced concepts... while having fun programming! 😊